

 Institución Universitaria	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	02
		Fecha	06-03-2019

1. IDENTIFICACIÓN DE LA GUÍA

Nombre de la guía:	Talleres evaluativos de Inteligencia Artificial
Código de la guía (No.):	006
Taller(es) o Laboratorio(s) aplicable(s):	Taller evaluativo en grupos (3 estudiantes) acerca de redes neuronales.
Tiempo de trabajo práctico estimado:	15 días (Entrega: Mayo 29 de 2019 antes 20 horas)
Asignatura(s) aplicable(s):	Inteligencia Artificial IA184-3
Programa(s) Académico(s) / Facultad(es):	Ingeniería de sistemas / Departamento de sistemas informacionales
Docente encargado	Carlos Alejandro Trujillo Anaya
Porcentaje evaluado de la asignatura	20%

COMPETENCIAS	CONTENIDO TEMÁTICO	INDICADOR DE LOGRO
El estudiante implementa redes neuronales artificiales con el objeto de resolver problemas de clasificación y regresión.	Redes Neuronales artificiales. <i>Backpropagation</i> . Funciones de activación. Optimizadores SGD, <i>Adam</i> <i>RMSPProp</i> . <i>Perceptrón</i> multicapa. Red neuronal convolucional. <i>Deep learning</i> .	Construye modelos descriptivos basados en redes neuronales tipo perceptrón y convolucionales para resolver problemas de aprendizaje supervisado. Implementa metodologías de ajuste (entrenamiento) de estos modelos y los valida empleando las métricas de desempeño de clasificadores y regresores.

2. FUNDAMENTO TEÓRICO

Las redes neuronales artificiales son una familia de algoritmos de aprendizaje de máquina que en una manera muy simplista emulan el funcionamiento de su contraparte biológica, con el fin de resolver problemas de aprendizaje supervisado y no supervisado. Las redes neuronales artificiales se han utilizado en una gran variedad de tareas, incluida la visión artificial, el reconocimiento de voz, la traducción automática, el filtrado de información en redes sociales, los videojuegos, el diagnóstico médico, entre muchos otros. Estos algoritmos construyen modelos discriminativos **tipo caja negra** empleando conexiones entre unidades que reciben y proveen valores reales, llamadas **neuronas**. Cada una de estas neuronas aporta al procesamiento de la información de entrada en términos de un coeficiente denominado el **peso**, donde éste y las excitaciones recibidas desde neuronas de capas anteriores son sopesados por una función de activación para generar la salida de la neurona. Dentro de las funciones de activación típicas se encuentran las funciones *ReLU*, *Sigmoide* y *tangente hiperbólica*: la no linealidad de los modelos construidos con las redes neuronales depende de estas funciones. De la misma manera, una capa de neuronas considera un coeficiente base, que no se conecta a

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	02
		Fecha	06-03-2019

neuronas de capas anteriores, pero que si puede alimentar a neuronas de capas posteriores llamado **bias**, el cual da cuenta de la respuesta de la capa cuando ésta no ha sido excitada. Tanto los *bias* de cada capa, como los *pesos* de las neuronas son ajustados durante el entrenamiento de la red. Las redes neuronales se entrenan empleando el algoritmo del **descenso del gradiente** o alguna de sus variantes como *ADAM* o *RMSPProp*. Estos algoritmos de optimización se emplean en el marco de algoritmos más generales denominados **backpropagation**. Los algoritmos de *backpropagation* hacen uso del concepto de regla de la cadena, para actualizar, desde la salida de la red hasta su entrada, el valor del peso de las neuronas en cada capa de acuerdo al negativo del valor del gradiente de la **función de costo** del modelo en un proceso iterativo de aprendizaje supervisado.

Dependiendo de la configuración, el tipo de capas empleadas, de las funciones de activación que se involucren, y mayormente, del modelo implementado, las redes neuronales se distinguen entre las redes más simples como los *perceptrones* o las más elaboradas como las *convolucionales*. El modelo más básico de red neuronal artificial incluye el modelamiento y el ajuste de una función anidada, donde cada una de las funciones interiores da cuenta de una capa de neuronas. En este modelo, todas las neuronas de una capa dada reciben únicamente información de las neuronas de una capa anterior en la estructura y alimentan únicamente a las neuronas de la capa posterior. Estas capas reciben el nombre de **fully-connected** y componen el denominado **perceptrón multicapa** en el que una capa de entrada (la cual recibe los datos a procesar), capas escondidas (que abstraen la información de los datos) y la capa de salida (que otorga el resultado del procesamiento) conforman completamente la estructura. Este tipo de estructura caracteriza al grupo de redes neuronales denominadas **feed-forward**, ya que, una vez la red está entrenada, la información se procesa desde la capa de entrada hasta la capa de salida sin retroalimentaciones. Una red neuronal puede llevar a cabo tareas tanto de regresión como de clasificación (las cuales son de interés en este curso), lo anterior dependerá *únicamente* de la capa de salida, en particular, de la función de activación y del número de neurona(s) que la componen. Normalmente para un problema de clasificación la capa de salida debe constituirse por un número de neuronas igual al número de clases que tenga el problema, las cuales tienen una función de activación *Sigmoide* con el fin de obtener salidas entre 0 y 1 (probabilidades de pertenencia a cada clase); por el contrario, para un problema de regresión, típicamente se tiene una única neurona en la capa de salida sin función de activación con el fin de obtener un número real sin rango fijo.

En la actualidad los algoritmos de **deep learning** están alcanzando los mejores resultados de predicción (tanto en regresión como en clasificación) en la mayoría de tareas que se pueden resolver con técnicas de aprendizaje de máquina. Estos algoritmos, es su definición más estricta, involucran a todas las redes neuronales artificiales con una número de capas escondidas mayor a tres, y donde algunas de ellas tienen estructuras distintas a aquella de la tradicional capa *fully-connected*. Dentro de esta subfamilia de algoritmos, una de los tipos de redes neuronales más utilizadas son las **redes neuronales convolucionales** o **CNNs** por sus siglas en inglés. Estas redes, dada su topología, están más orientadas a las tareas que involucran imágenes, sin embargo se han reportado aplicaciones de las mismas en distintos campos de las ciencias computacionales. Debido a que la naturaleza *fully-connected* del *perceptrón* lo hace propenso al *overfitting*, las CNN se pueden entender como versiones regularizadas de los mismos con el fin de evitar dicho inconveniente. Lo anterior lo llevan a cabo aprovechando el patrón jerárquico de los datos, al ensamblar patrones más complejos haciendo uso de patrones más pequeños y más simples. De allí que en términos de complejidad de las conexiones, las CNN se encuentran en el extremo inferior respecto a los *perceptrones*. La capa convolucional es el bloque central de una CNN. Los parámetros de la capa consisten en un conjunto de **filtros** con campos receptivos pequeños, pero que se aplican a lo largo de toda la profundidad del volumen de entrada de datos. Durante el procesamiento de una nueva instancia para clasificar o regresar, cada filtro aplica una convolución en todas las dimensiones que tengan los datos de entrada, calculando el producto punto entre las entradas del filtro y los datos, produciendo de esta forma un mapa de activación bidimensional de ese filtro. Como resultado, la red aprende ajustando los valores de los *filtros* para que se activen cuando detecten algún tipo específico de característica

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	02
		Fecha	06-03-2019

en alguna posición espacial de los datos de entrada. Las CNN utilizan relativamente poco o nada de pre-procesamiento de los datos en comparación con otros modelos predictivos. Lo anterior implica que la CNN, en sus etapas convolucionales, se comporta como un extractor de características automático. Por consiguiente, la posibilidad de hacer implementaciones robustas con un conocimiento previo superficial del tema por parte del desarrollador ha propulsado, así como sus impresionantes desempeños, la extensión de la aplicabilidad del *deep learning* en distintas tareas de las ciencias computacionales y otras ramas del conocimiento.

3. OBJETIVO(S)

- Implementar en un regresor multivariado empleando un perceptrón multicapa.
- Implementar un clasificador binario y uno multiclase empleando perceptrones multicapa.
- Implementar un clasificador de *deep learning* para un problema de clasificación complejo.
- Emplear métricas de validación de desempeño de redes neuronales y comparar su desempeño con otros modelos predictivos.

4. RECURSOS REQUERIDOS

- Un computador con un compilador de Python actualizado. La librería *Keras* debidamente instalada. Se recomienda contar con la versión que corre en GPU.
- Un computador con un editor de texto para la redacción del informe.

5. PROCEDIMIENTO

Del repositorio UCI (<https://archive.ics.uci.edu/ml/datasets.php>) escoja tres *datasets*:

- Un *dataset* de regresión.
- Un *dataset* de clasificación binaria.
- Un *dataset* de clasificación multiclase.

En la medida de lo posible, los *datasets* seleccionados deben ser los que ya ha trabajado en el curso. Cada equipo de trabajo debe escoger *datasets* distintos.

- 1) Para el *dataset* de regresión, implemente un perceptrón multicapa que le permita predecir el valor de la variable de salida empleando como entradas TODAS las características del *dataset*. Justifique los valores asignados a los *hiperparámetros* de su modelo, así como el optimizador seleccionado para entrenar su red. Valide el desempeño de su red neuronal empleando alguna métrica cuantitativa de desempeño de regresión. Compare sus resultados con aquellos que se pueden obtener empleando una regresión lineal para el mismo *dataset*. Justifique sus resultados. Recuerde emplear conjuntos de entrenamiento y conjuntos de validación distintos.
- 2) Para el *dataset* de clasificación binario, implemente un perceptrón multicapa que le permita predecir la clase de salida empleando como entradas TODAS las características del *dataset*. Justifique los valores asignados a los *hiperparámetros* de su modelo, así como el optimizador seleccionado para entrenar su red. Valide el desempeño de su red neuronal empleando alguna métrica cuantitativa de desempeño de clasificación. Compare sus resultados con aquellos que se pueden obtener empleando un clasificador básico (regresor logístico, Naive Bayes, o k-NN) para el mismo *dataset*. Justifique sus resultados. Recuerde emplear conjuntos de entrenamiento y conjuntos de validación distintos.

 Institución Universitaria	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	02
		Fecha	06-03-2019

- 3) Para el *dataset* de clasificación multiclase, implemente un perceptrón multicapa que le permita predecir la clase de salida empleando como entradas TODAS las características del *dataset*. Justifique los valores asignados a los *hiperparámetros* de su modelo, así como el optimizador seleccionado para entrenar su red. Valide el desempeño de su red neuronal empleando alguna métrica cuantitativa de desempeño de clasificación. Compare sus resultados con aquellos que se pueden obtener empleando un clasificador básico (regresor logístico, Naive Bayes o k-NN) para el mismo *dataset*. Justifique sus resultados. Recuerde emplear conjuntos de entrenamiento y conjuntos de validación distintos.
- 4) Descargue el *dataset* **8-scenes** que se encuentra en el siguiente link (<https://drive.google.com/drive/folders/1AE77zHu1sNpVI3-xLzdZK6jGLKFJngmr?usp=sharing>). Igualmente encontrará un script de Python que le sirve como base para la elaboración de este punto. Implemente una red neuronal convolucional que le permita clasificar adecuadamente las imágenes de este *dataset*. Justifique los valores asignados a los *hiperparámetros* de su modelo, así como el optimizador seleccionado para entrenar su red. Valide el desempeño de su CNN empleando el *accuracy* sobre las imágenes de validación. Justifique sus resultados. Los conjuntos de entrenamiento y conjuntos de validación vienen establecidos en el link, usted no los tiene que construir. El reto en este punto consiste en obtener un *accuracy* por encima del 95%.

6. INFORME Y SUSTENTACIÓN

El informe debe incluir las respuestas a las preguntas suministradas en esta guía así como los argumentos solicitados. No se extienda en teoría general, solo utilice aquella que le ayude a justificar sus respuestas.

La sustentación del código enviado se hará en el aula de clase. Debe tener conocimiento de cada una de las estrategias implementadas ya que la sustentación incluirá conocimiento teórico. Todos los integrantes del grupo de trabajo deben prepararse para la sustentación del trabajo. La sustentación corresponderá a una nota **individual**. Si no asiste a la sesión de sustentación, se le asignará un 0.0 en la nota de este trabajo.

7. REFERENCIAS

- Andriy Burkov, "The hundred-page machine learning book", chapter 6.
- Shai Shalev-Shwartz and Shai Ben-David "Understanding Machine Learning: From Theory to Algorithms", chapter 20.
- <https://machinelearningmastery.com/regression-tutorial-keras-deep-learning-library-python/>
- <https://machinelearningmastery.com/tutorial-first-neural-network-python-keras/>

Elaborado por:	Carlos Alejandro Trujillo Anaya
Revisado por:	Carlos Alejandro Trujillo Anaya
Versión:	001
Fecha:	Mayo 15 de 2019